mb-free___transport-eager-32768

An AgentMPI run analysis

Abstract

This is the far end of the eager column in a ten-cell transport sweep, and like the cell below it, it did not complete. `results/microbench/free-transport.json` records `completed: false`, `n_failed: 8` and `ERR_CONTEXT_OVERFLOW` for eight agent-free ranks asked to ring-exchange a 32,768-word payload under `Mode.EAGER`. The trace has 18 events instead of 34 — eight `rank.init`, eight `rank.finalize`, and the two job markers — with no `msg.send` and no `msg.recv`, an empty `messages` table and no communication matrix to draw. What distinguishes this cell from its 8,192-word neighbour is that it is over *both* limits at once. Its 43,116 estimated tokens exceed the 4,096-token unexpected-message credit by 10.5×, which is what actually refused it, and they also exceed the receiver's entire 32,768-token context budget by 1.32×, so no amount of credit would have saved it: with an unlimited budget the receive would have raised `ERR_TRUNCATE` instead. This is the size at which eager is not merely over-provisioned but incoherent — the message does not fit in the receiver. The rendezvous cell at the same size moved the identical blob and finished with zero context consumed.

1 What this run is

property	value
run	`mb-free__transport-eager-32768`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`none` × 8
events	18
wall time	0.07 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	0	0 eager, 0 rendezvous
tokens sent	0	0 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	$0.0000	0.0000 $/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.

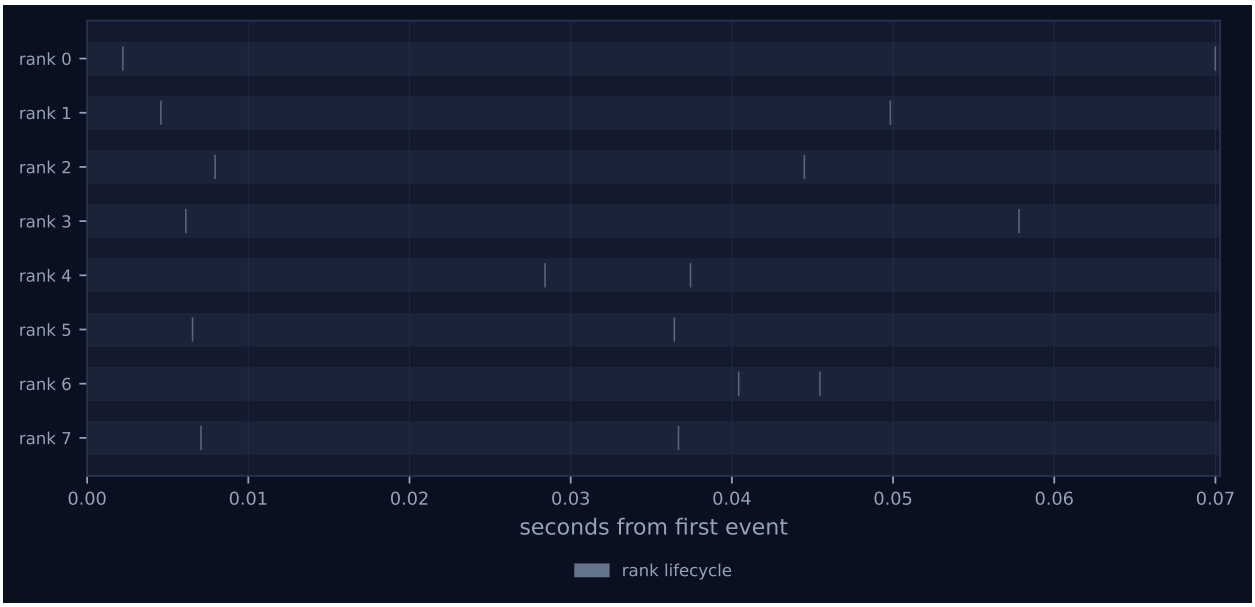

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	0	0	0	0.0	finalized
1	0.00	0.0	0	0	0	0	0	0.0	finalized
2	0.00	0.0	0	0	0	0	0	0.0	finalized
3	0.00	0.0	0	0	0	0	0	0.0	finalized
4	0.00	0.0	0	0	0	0	0	0.0	finalized
5	0.00	0.0	0	0	0	0	0	0.0	finalized
6	0.00	0.0	0	0	0	0	0	0.0	finalized
7	0.00	0.0	0	0	0	0	0	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 shows eight lanes with two grey lifecycle diamonds each and nothing between them. The viewer's legend lists only "rank lifecycle 16"; the green messages swatch that appears in every completed cell of this sweep is absent, because there is no message event of any kind. `MESSAGES 0`, `TOKENS DEFERRED 0.0k`, 0% context occupancy on every rank, and every rank's state `finalized`. There is no `figures/comm.pdf` for this run and no `fig:comm` reference in the generated section, because a matrix of zero messages is not a picture.

The span deserves attention: 0.07 s, the longest of any eager cell in the sweep, including the three that actually delivered messages — 0.019 s, 0.032 s and 0.035 s at 128, 512 and 2,048 words. A run that sent nothing took twice as long as the largest run that sent everything. The cause is in the order of operations inside `Communicator.send`: `fabric.blobs.put(payload)` runs first, before `_choose_mode` and before the credit check, so each rank serialised, hashed and wrote its 163,840-byte payload to the content store and only then had the send refused. The blob is on disk, at `runs/mb-free/transport-eager-32768/blobs/04/af1d57...`, 163,840 bytes, and it carries the same digest as the blob written by the rendezvous cell at this size — the two runs put byte-identical content into their fabrics. Nearly all of this run's 70 ms is that write, spread over a 38 ms rank start-up window. The lanes in Figure 1 are wider apart than in any other eager cell for that reason and for no protocol reason at all.

7 Interpretation

The failure path is the same as the 8,192-word cell's and can be pinned down the same way. `ERR_CONTEXT_OVERFLOW` is `AmpiContextOverflow`, which in a run with no agent calls can only come from `Communicator._await_eager_credit`. That function has two raising branches: an immediate

precondition when a single payload exceeds the destination's `unexpected_limit`, and a deadline expiry inside the credit-polling loop. The benchmark's timeout is `stall_timeout` = 12 s, and the entire job spans 0.07 s with the per-rank figure in the results file at 0.01 s, so the loop was never entered; there is also no `transport.credit_stall` event, which the loop emits the first time it waits. It was the precondition: 43,116 tokens against a limit of 4,096. The empty `messages` and `seqs` tables show the raise preceded the row insert and even the sequence-number increment, so the ring did not form one edge.

What this cell adds beyond its neighbour is the second ceiling. At 8,192 words the payload was 10,779 tokens: over the 4,096-token credit, but comfortably inside the 32,768-token context budget, so that cell isolates the flow-control limit and shows AgentMPI refusing a message the receiver could have held. Here the payload is 43,116 tokens, which is 1.32× the receiver's whole context window. Even with infinite eager credit this exchange is impossible: the message would have arrived, `_deliver` would have called `rt.admit`, `ContextAccount.admit` would have found `would_fit` false against a budget of 32,768 with `strict_context` true, and the run would have failed with `ERR_TRUNCATE` rather than `ERR_CONTEXT_OVERFLOW`. The two failed cells therefore say different things. The 8,192-word one says eager has a flow-control ceiling below the capacity ceiling; this one says that above the capacity ceiling eager is not a transport choice at all, because there is nowhere for the payload to go. That is the sharpest statement of the sweep's premise — an agent's context is a bounded, exhaustible resource — and the sharpest possible contrast with its rendezvous twin, `mb-free__transport-rendezvous-32768`, which moved this same 43,116-token artefact through eight envelopes in 34 events with zero admissions, zero context high water and every rank at 0% occupancy. A payload larger than the receiver's entire context window crossed the ring without consuming any of it.

One defect is worth recording, and it belongs to AgentMPI rather than to the benchmark. `_await_eager_credit`'s precondition branch raises without emitting anything, while the branch below it emits `transport.credit_stall` on entry and `transport.credit_granted` on release. The result is that a run in which every one of eight ranks failed is recorded as `ok: true` with `failed_ranks: []` in `job.finish`, as `degraded: false` with zero findings in `metrics.json`, and as `FAILURES 0` in the viewer, with all eight ranks `finalized`. The benchmark contributes to this — its `rank_main` catches `AmpiError` and returns a dictionary, so the runtime never sees a failure — but the harness cannot log an event the library does not emit, and the asymmetry between the two rejection paths is the library's. It is not a correctness bug: the refusal is right and its message names the remedy. It is an observability bug, and it is the reason this analysis has to reach outside the trace for the one fact that matters most about the run.

8 Threats to this reading

The failure is not in the trace. Everything asserting that this run failed — `completed: false`, `n_failed: 8`, the error class — comes from `results/microbench/free-transport.json`, and the trace supports it only negatively, through 18 events instead of 34 and two empty fabric tables. The claim that the precondition branch fired rather than the stalling branch is likewise an inference from a 0.07 s span against a 12 s timeout and from an absent event. The counterfactual in the second paragraph — that with unlimited credit this cell would have failed with `ERR_TRUNCATE` at admission — is read off the code path in `rank.py` and was not run; it is the most speculative claim in this document, though the arithmetic behind it, 43,116 against 32,768, is not in doubt. Token counts are estimates: provenance records `tokenizer_exact: false`, and 43,116 is $\lceil 163840/3.8 \rfloor$

from a character heuristic applied to one repeated word. A real tokeniser would price this text nearer 32,768 tokens, which would put it essentially at the context budget rather than 32% above it and would make the second-ceiling argument marginal instead of clear-cut — the first ceiling, at 8× the credit either way, would be untouched. Both budgets are benchmark parameters rather than defaults (`unexpected_limit` 4,096 and `context_budget` 32,768, against shipped defaults of 8× the eager limit and 128,000), so this run shows where AgentMPI puts a ceiling when told to, not where the ceiling is in practice. Finally, the executor is `none` with zero agent calls, so nothing here concerns agent behaviour, and the wall time is dominated by a 160 KiB blob write and eight SQLite-backed rank start-ups.